



POLITECNICO
MILANO 1863

Streaming ML VS Continual Learning

Federico Giannini

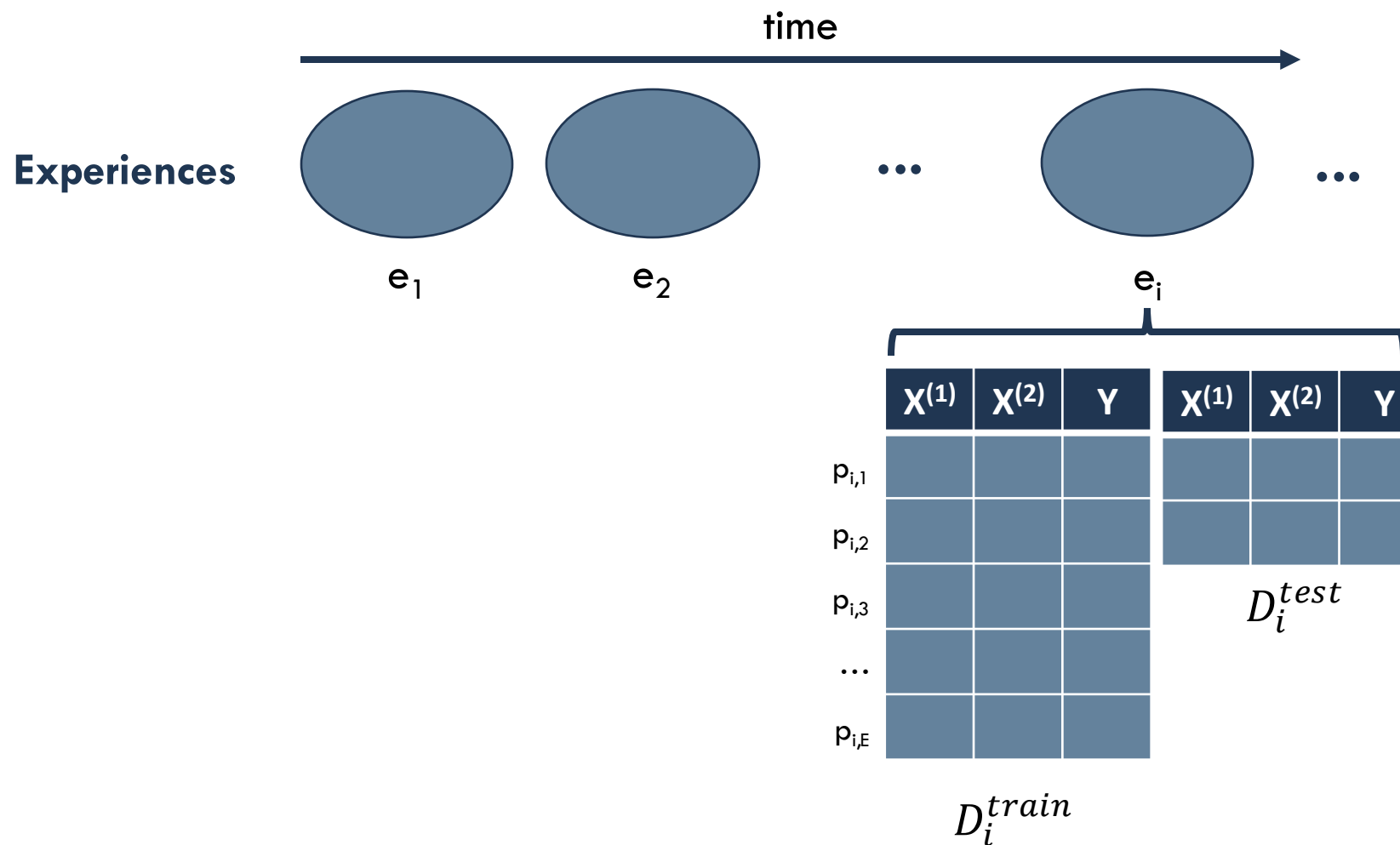
Ph.D. Student

DEIB – Politecnico di Milano

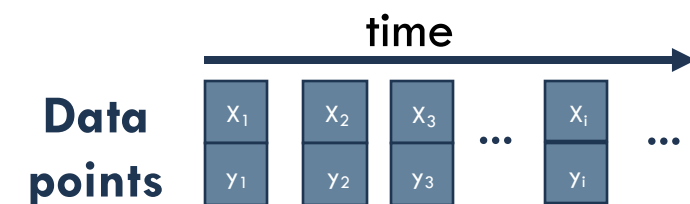
federico.giannini@polimi.it

SML vs CL: Different Settings

CONTINUAL LEARNING



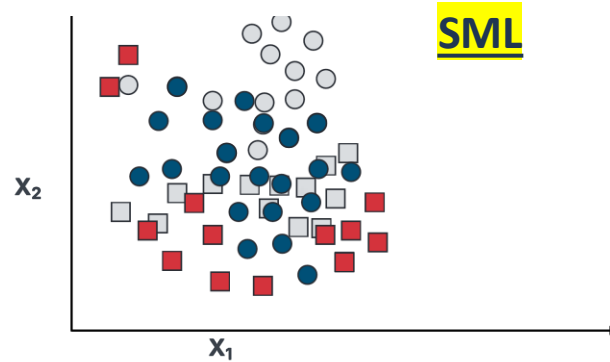
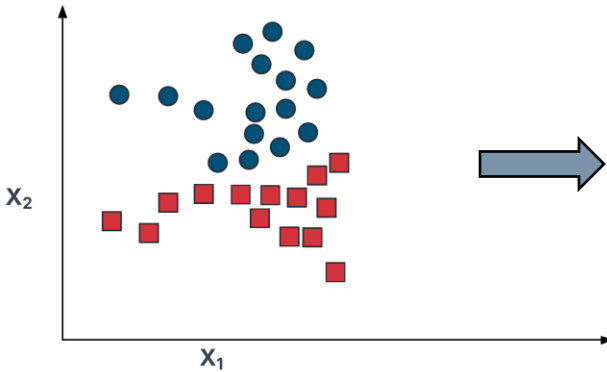
STREAMING ML



- ▶ Single data points or mini-batches arrive over time.
- ▶ No distinction between train and test (Prequential evaluation).

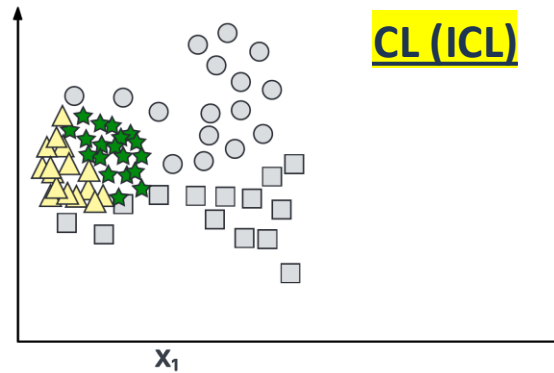
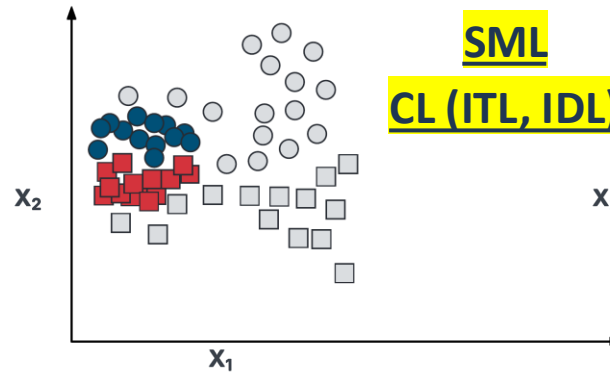
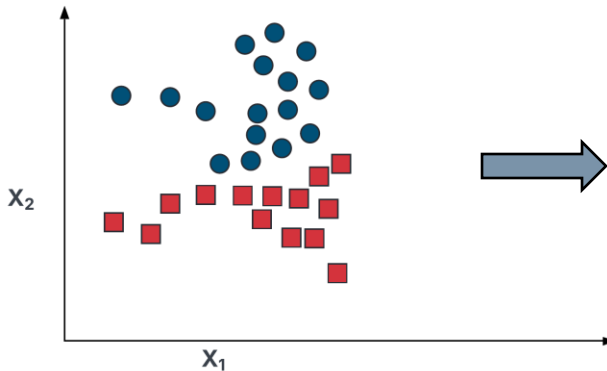
SML vs CL: different types of drifts

Original distribution

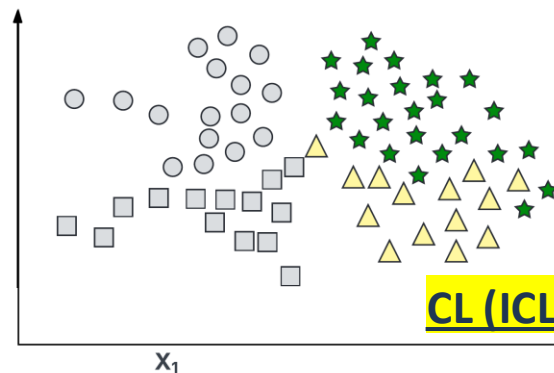
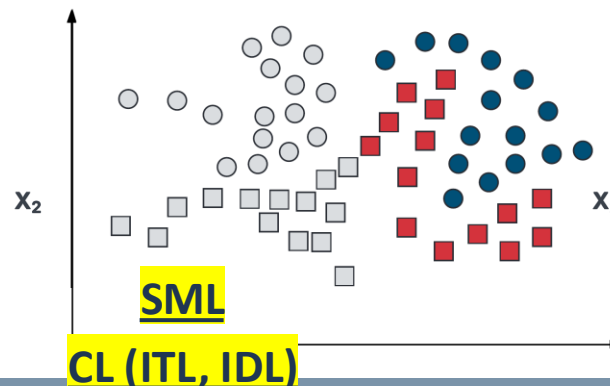


The new concept changes the classification rules w.r.t. to the same feature space.

ICL: Incr. Class Learn.
ITL: Incr. Task Learn.
IDL: Incr. Domain Learn.



The new concept introduces a new input distribution. The classification rules for previous instances do not change.



SML and CL: different goals

CONTINUAL LEARNING

- After a concept drift, integrate new knowledge without forgetting the earlier one (it assumes the new concept does not contradict the previous).
- Achieve a trade-off between plasticity and stability.
- Perform well in all current and previously observed experiences.

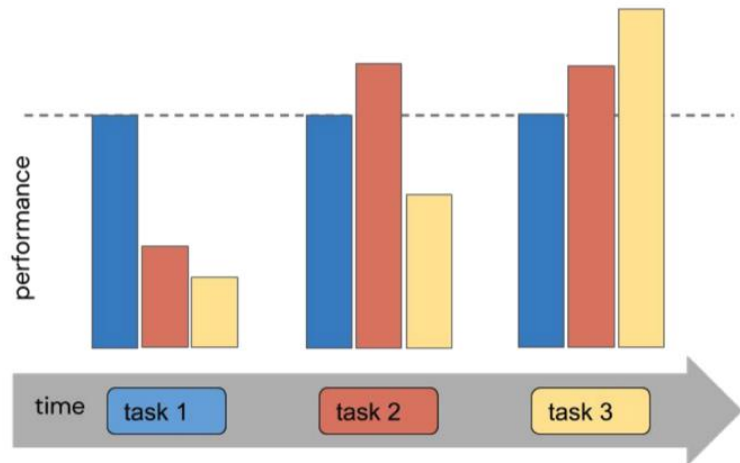
STREAMING ML

- Quickly adapt to new distribution.
- Perform well on the current concept and ignore the previous ones (since it may contradict them).
- If a previously learned concept reappears in the future, the model will relearn it.
- Develop an online classifier that can respond at any time.



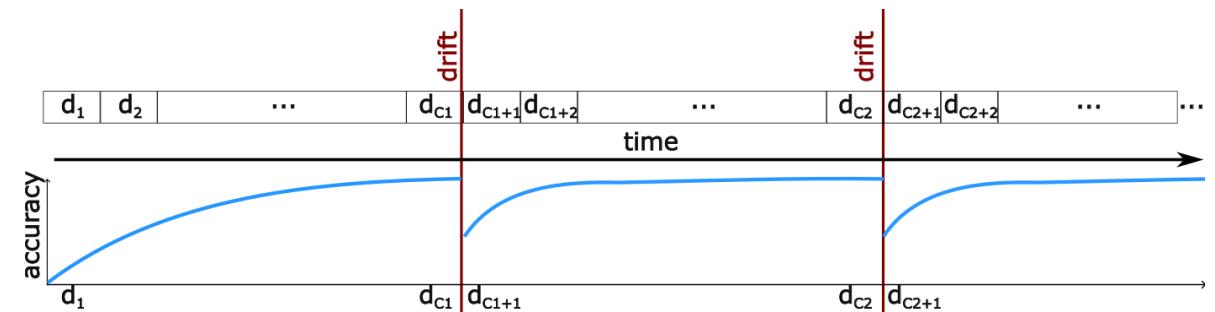
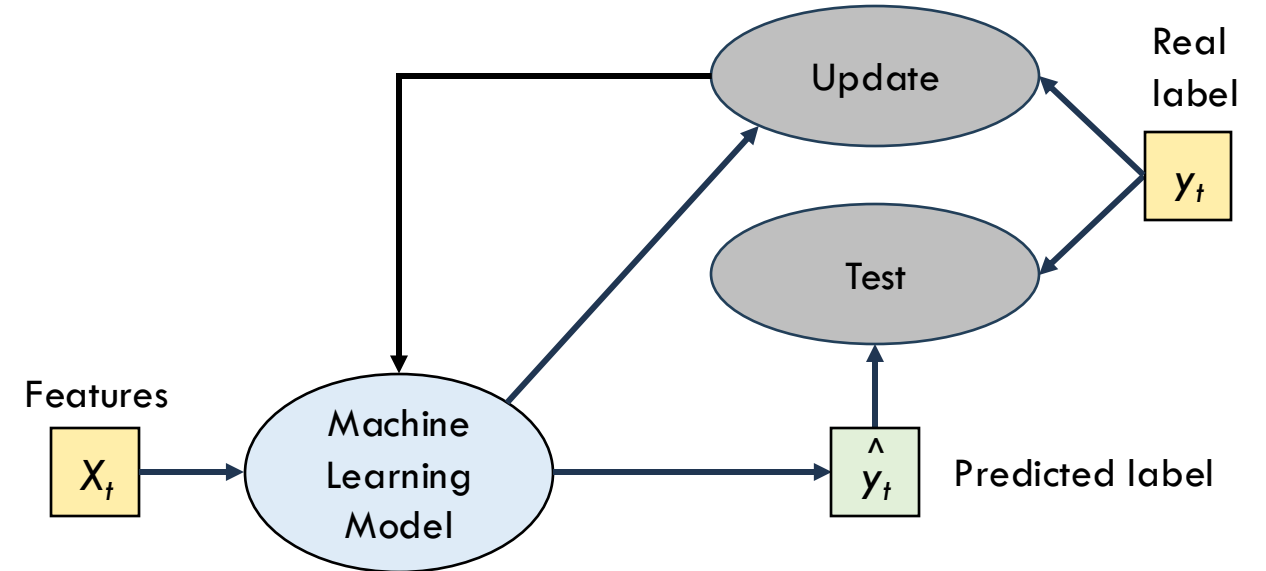
SML vs CL: Different Evaluations

CONTINUAL LEARNING



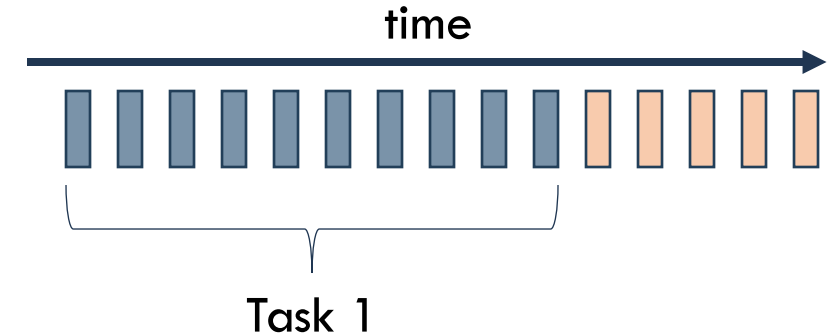
	Test e_1	Test e_2	Test e_3
Train e_1	$Acc_{1,1}$	$Acc_{1,2}$	$Acc_{1,3}$
Train e_2	$Acc_{2,1}$	$Acc_{2,2}$	$Acc_{2,3}$
Train e_3	$Acc_{3,1}$	$Acc_{3,2}$	$Acc_{3,3}$

STREAMING ML



Online CL: an emerging paradigm that approaches an online setting

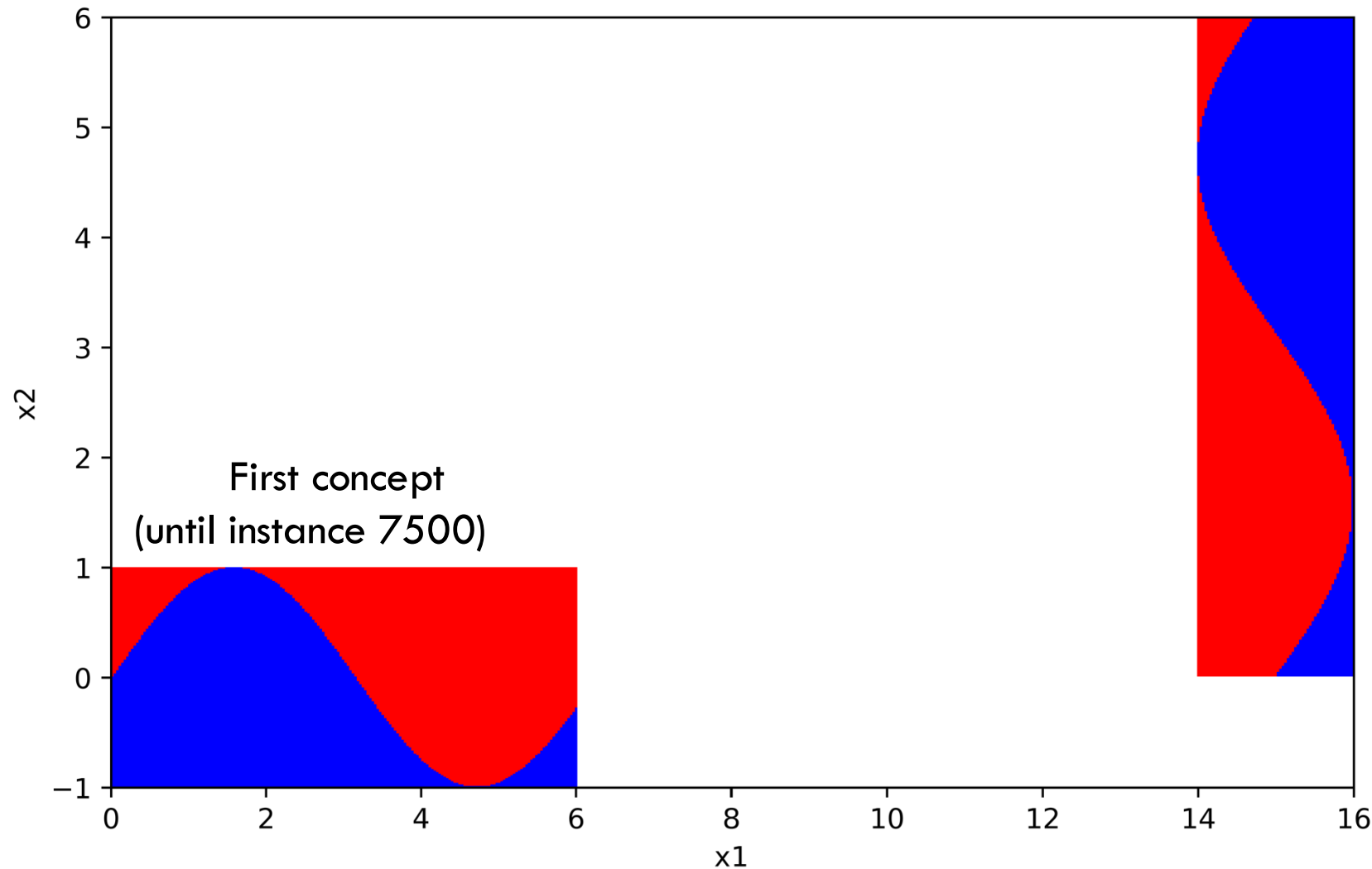
- Not all the training data points associated with a task are provided at once.
- The model receives a mini-batch at each timestamp with tens of data points.
- Test sets are set aside before the data stream starts.
- The model must perform one epoch of training each time it receives a minibatch.
- Mini-batches are not revisited over time.
- It is applied to ICL and IDL. Some IDL settings remove the supervised information about the drift (task-free OCL).
- Strategies must be adapted to be performed online.
- It just changes the data stream velocity. OCL's goal is aligned with CL: learning new knowledge while avoid forgetting on previous tasks, in a setting where drifts are not contradictory.



Let's introduce some examples to compare SML and CL in different settings!



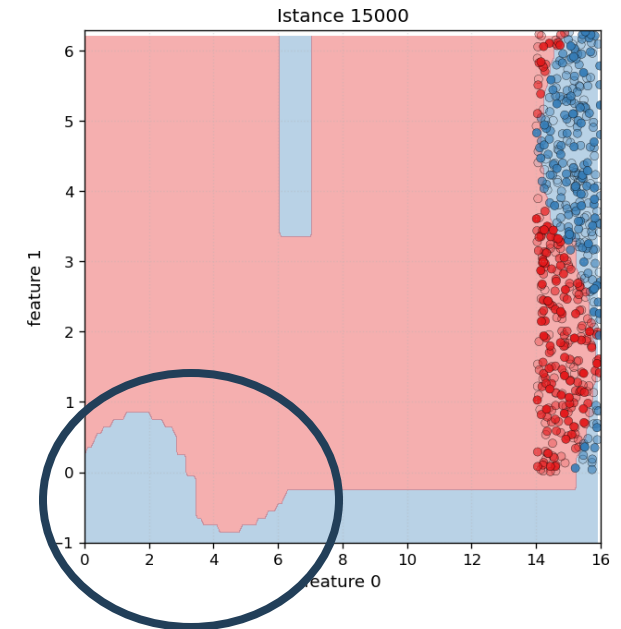
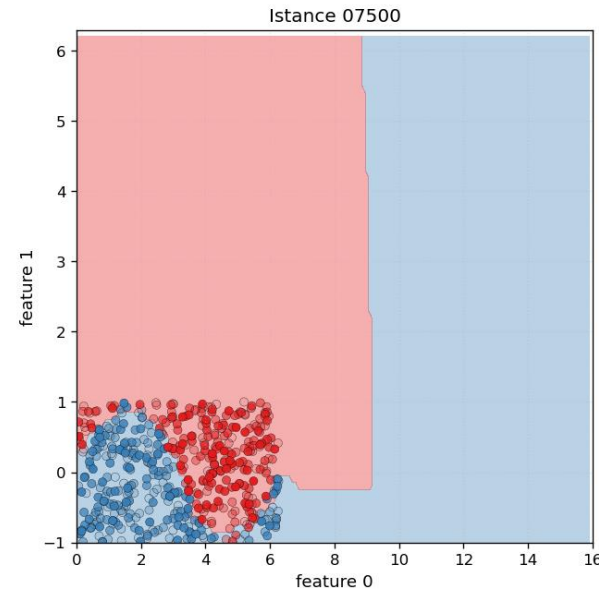
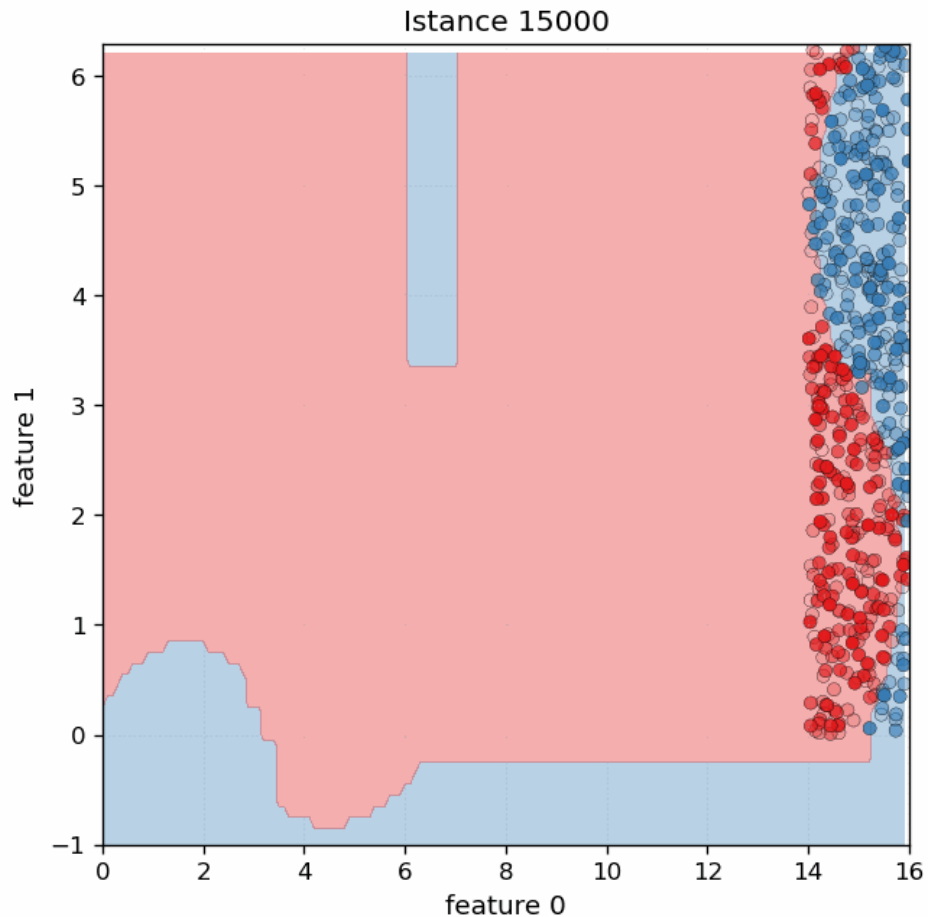
First example: SINE



Second concept
(from instance 7500
onwards).

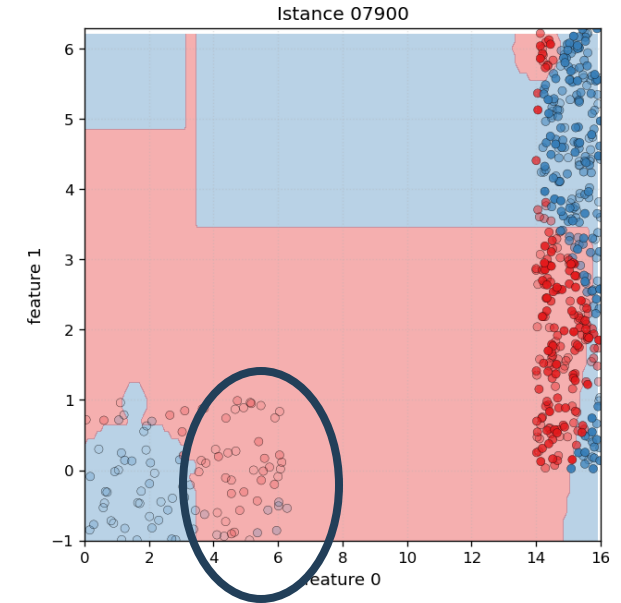
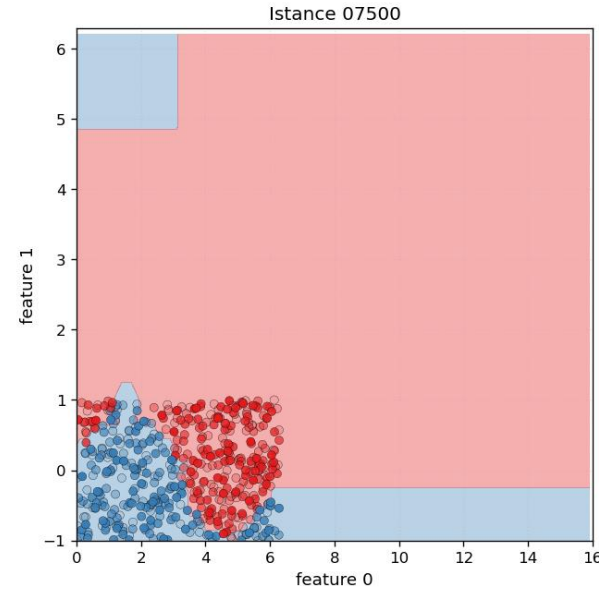
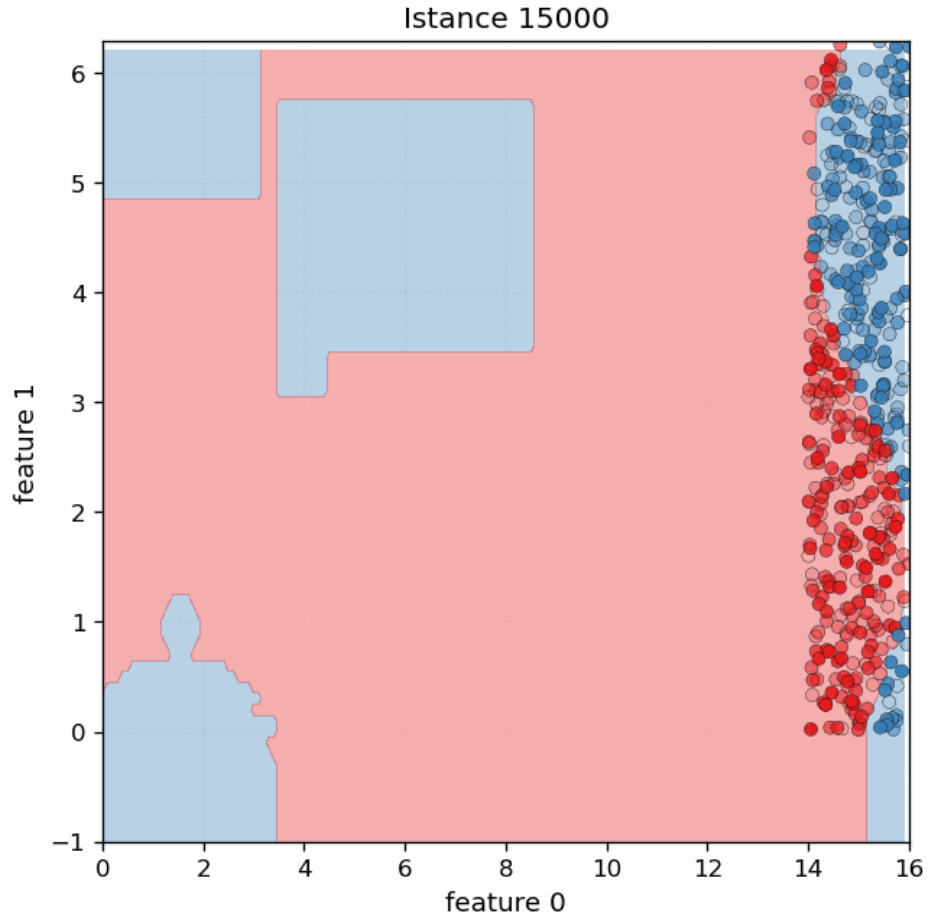
It does not contradict the
first, since it is defined in
another feature subspace

Let's train a Hoeffding Tree (HT)



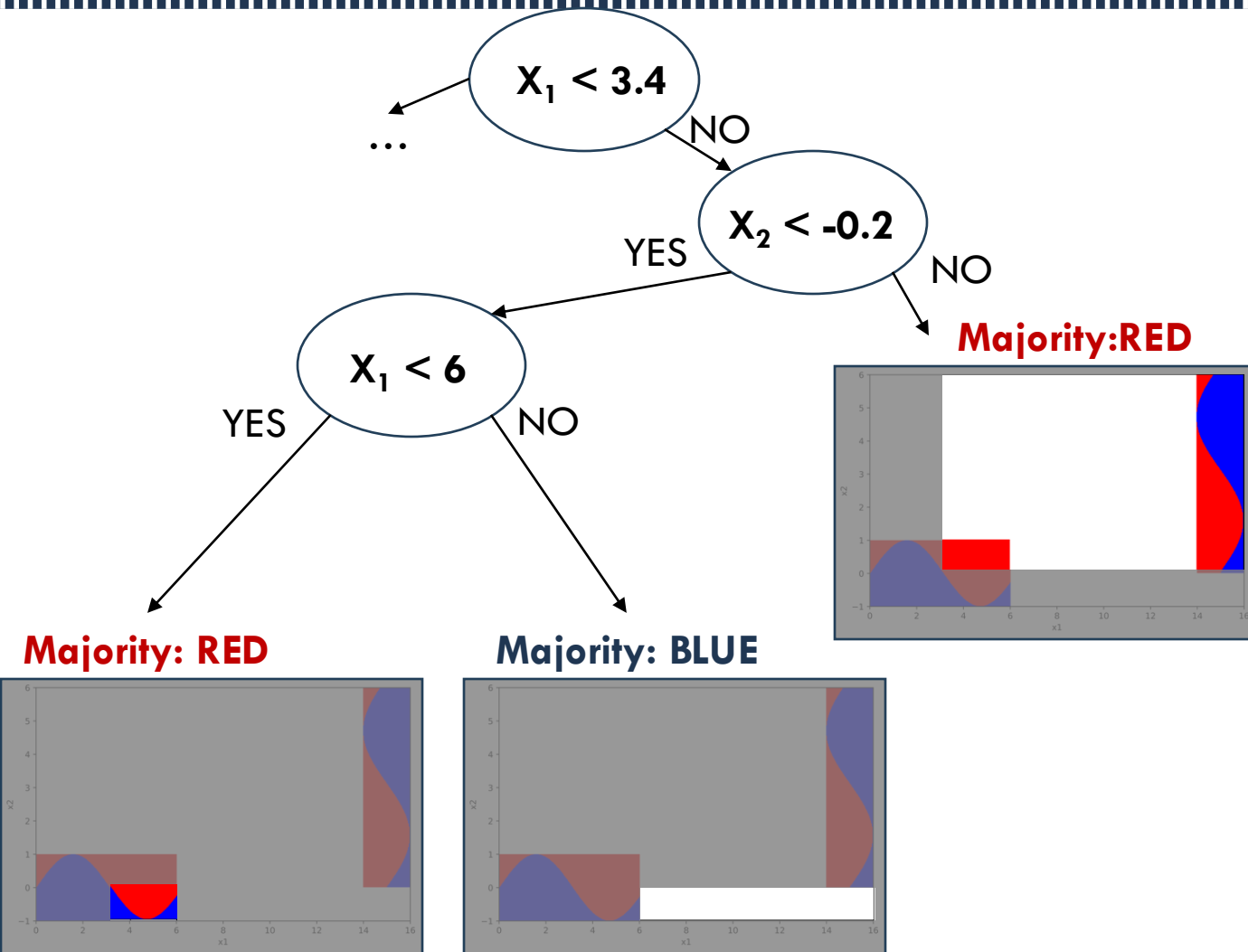
It seems not to forget the first concept.
But HT is not an adaptive model.

Let's now train a Hoeffding Adaptive Tree (HAT)

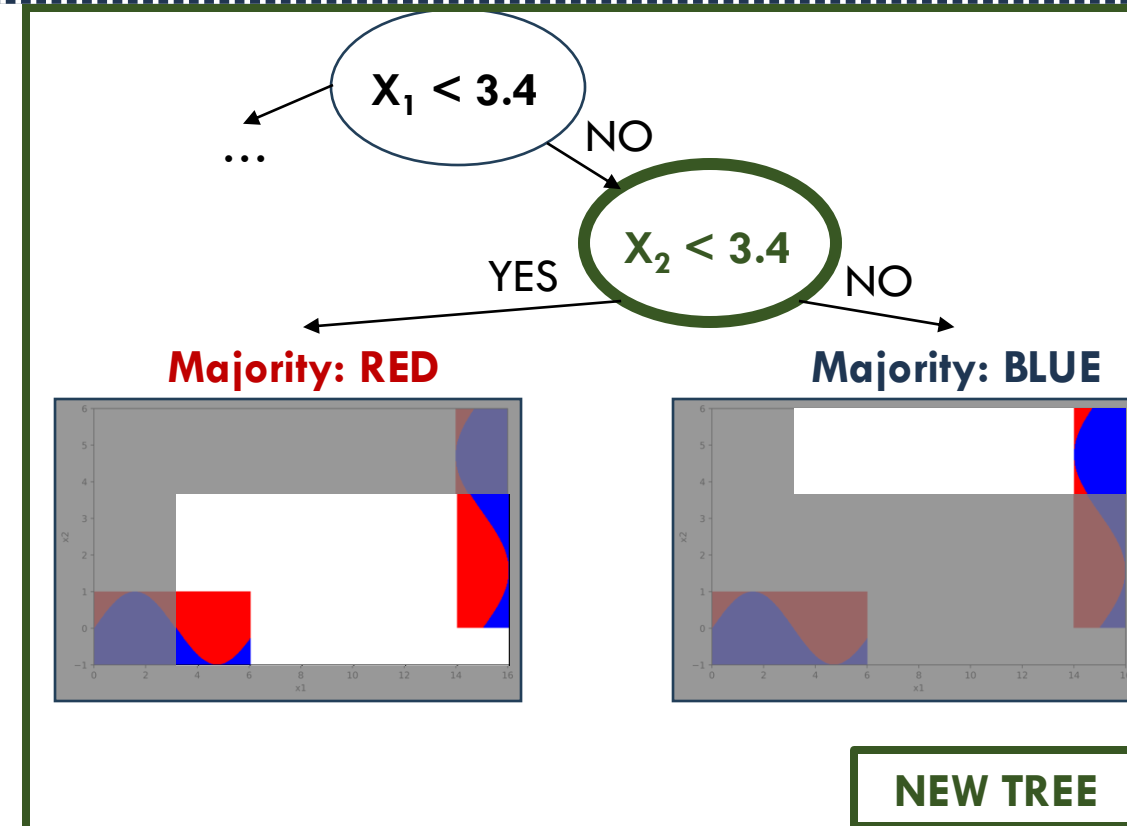
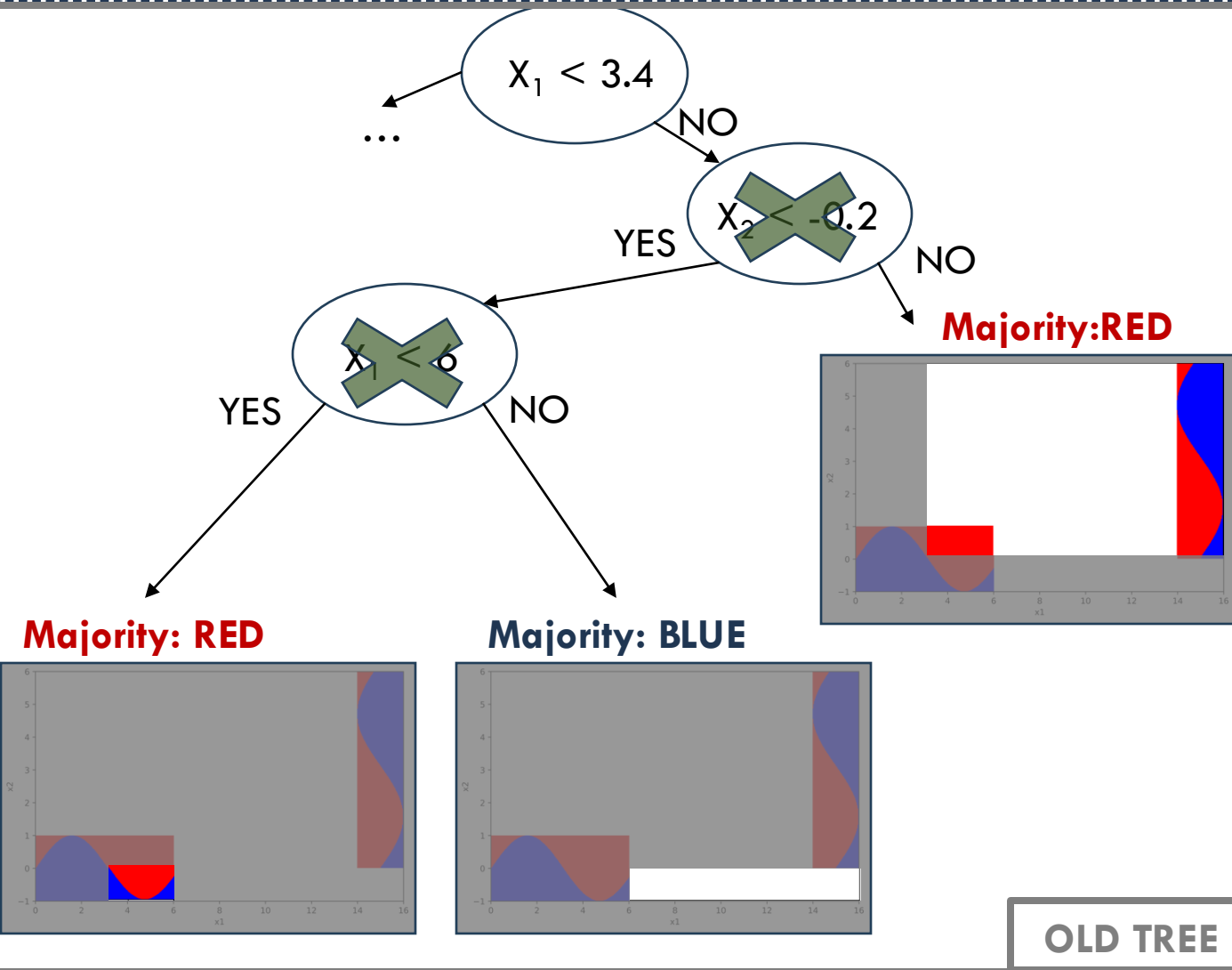


It forgets the first concept.

Let's have a look into the HAT's decision tree:



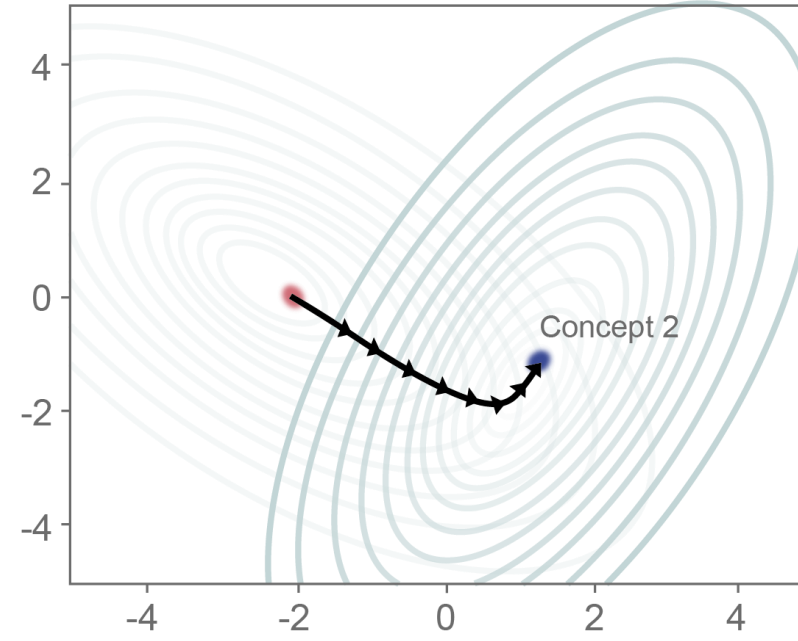
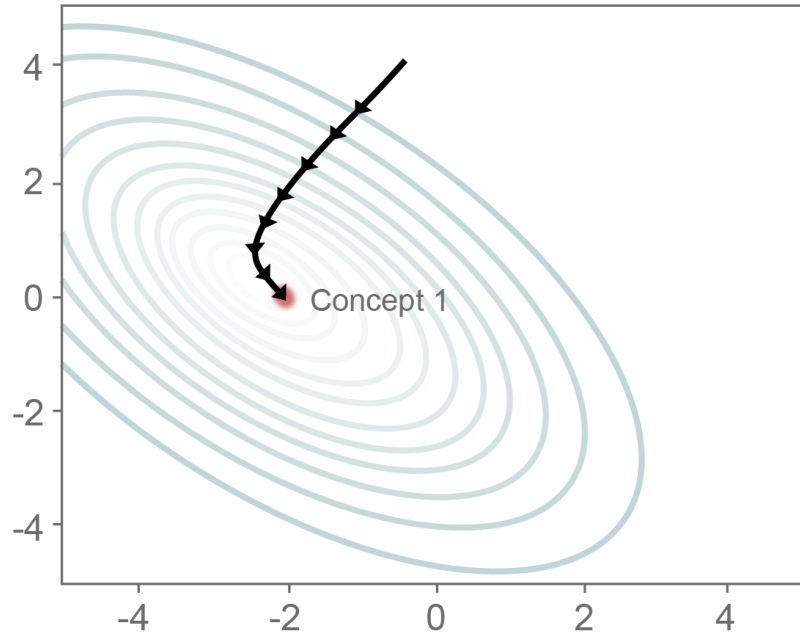
Let's have a look into the HAT's decision tree:



The adaptive mechanism of HAT substitutes the subtree with a new one that overwrites the previous concept.

Being adaptive causes forgetting!

But, when trying to avoid forgetting after a drift introducing a contradiction



It's not possible to jointly optimize both concepts together:

- One cannot produce correct predictions for all the tasks (tasks are contradictory).
- Learning cannot converge.

Avoiding forgetting after drifts introducing a contradiction

- Forgetting assumes a different meaning: a task may recur in the future, or the previous knowledge may help boost adaptation to new tasks. Remembering past tasks may solve these issues.
- One can build a cumulative general knowledge over the entire data stream and have specific model parameters for each task.
- When learning the new knowledge, the model can select the portion of the cumulated knowledge that is useful for the current concept.
- Some CL architectural strategies isolate a subpart of the network for each task and utilize transfer learning to reuse a portion of previously acquired knowledge for solving the new task.



Second example: MNIST

Let's now move to a more concrete example!

We want to test SML models and CL strategies in two different scenarios:

- **Input drift scenario (CL-like):** a drift introduces a new distribution specified in a new feature space. We do not want to forget the previous tasks/concepts.
- **Contradictory drift scenario:** a drift introduces a change in the class boundary. The input distribution remains fixed.

All drifts are abrupt.



MNIST dataset

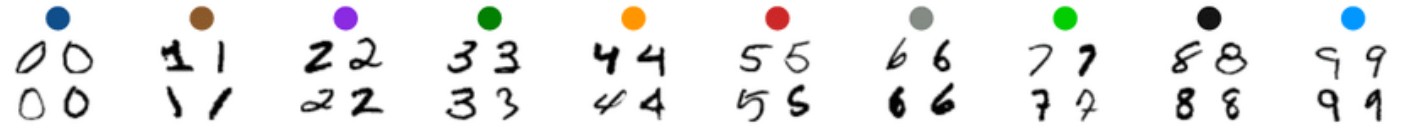
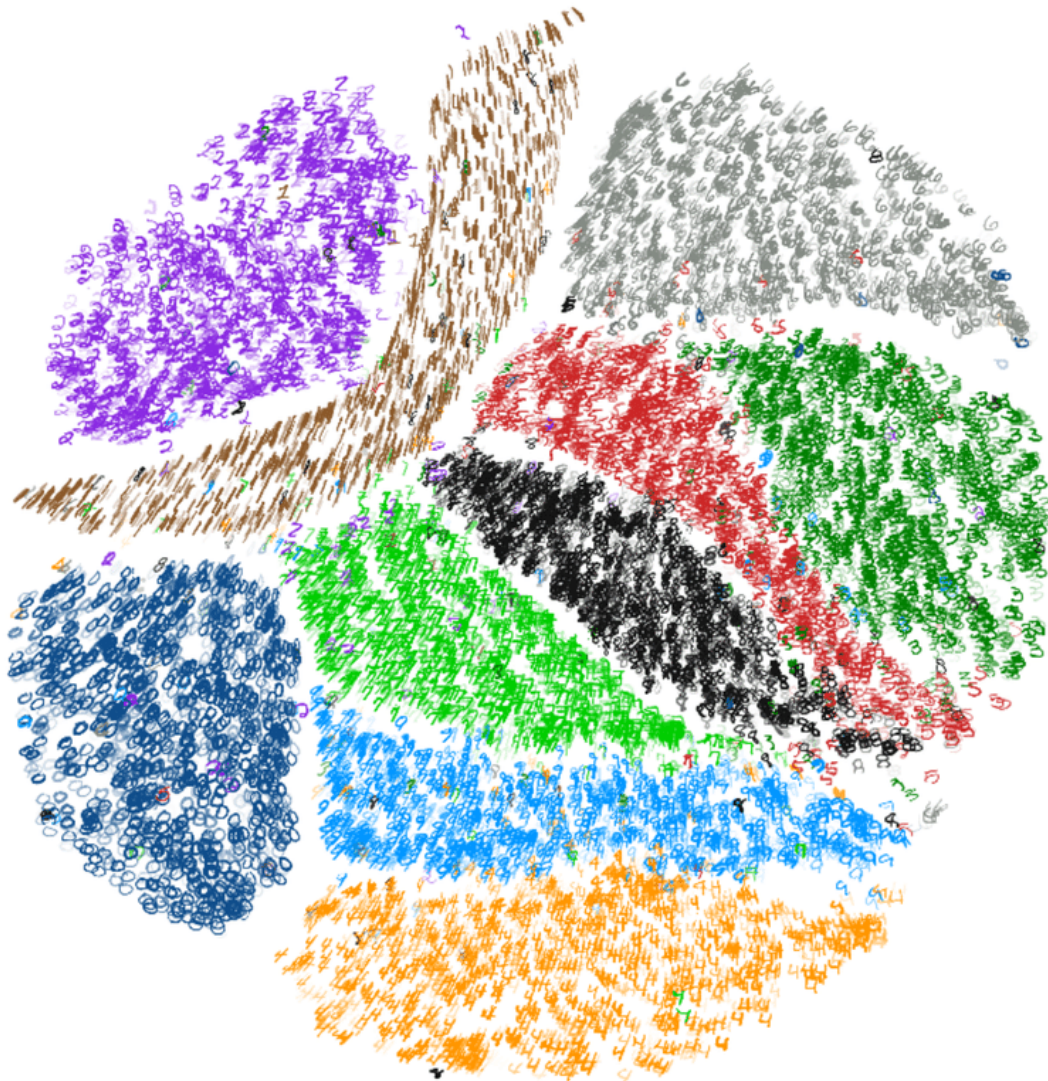


10 different classes of handwritten digits.

Shape: 28x28 in greyscale.

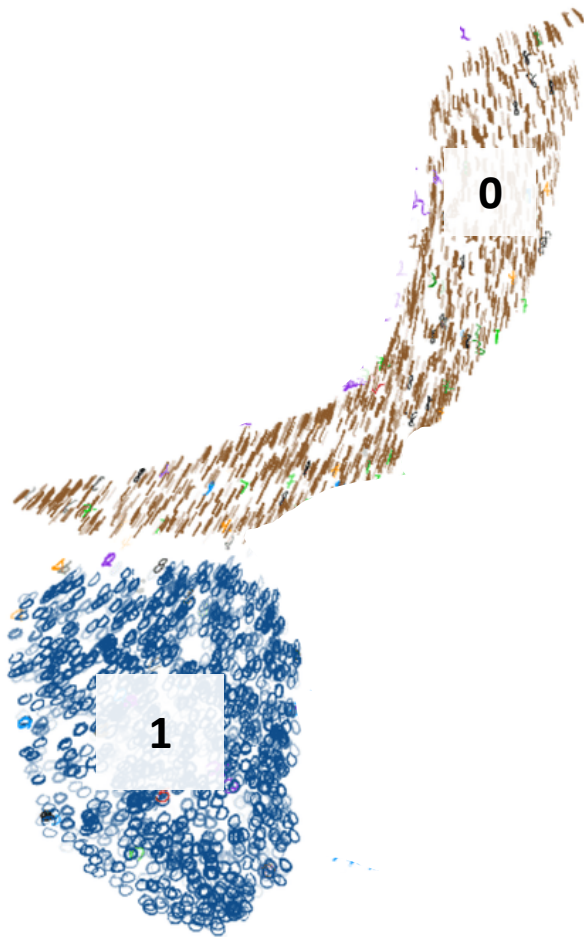
The goal is usually to recognise the digits.

MNIST dataset



- Let's use t-SNE as a visualization tool: it allows us to project the data into two dimensions and clearly show how each digit class in the MNIST dataset tends to occupy its own subspace. This separation already exists in the original high-dimensional images, but it is more difficult to observe directly because it is more fragmented and dispersed.
- CL typically breaks down the problem into tasks. Each task contains two digits.
- Each task introduces a subproblem.

MNIST dataset: Input drift scenario (CL-like)



1st task

2nd task

3rd task

4th task

5th task

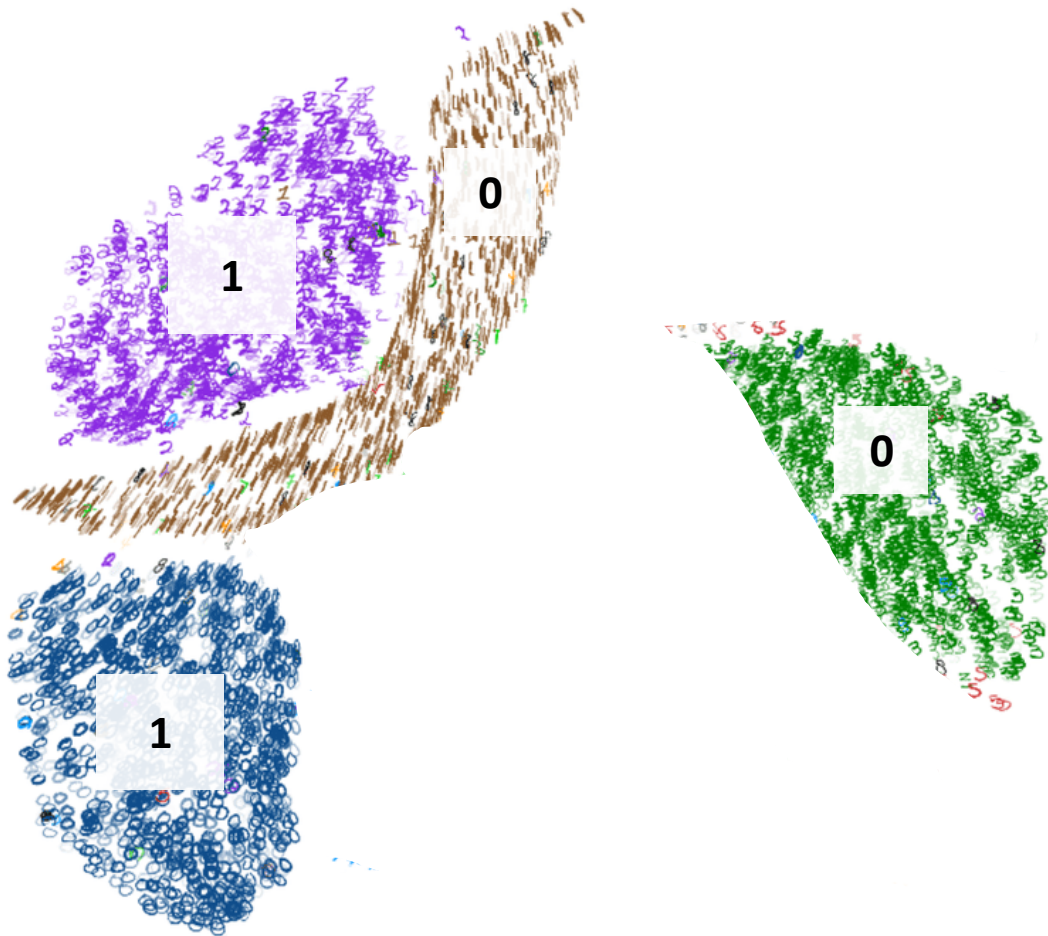
We consider a binary classification problem: **even/odd**.

Each task introduces a new couple of digits.

What we learned during previous tasks is still valid.

It's an Incremental Domain Learning Scenario.

MNIST dataset: Input drift scenario (CL-like)



 00 00	 11 11	 22 22	 33 33		
1 st task		2 nd task		3 rd task	4 th task
					5 th task

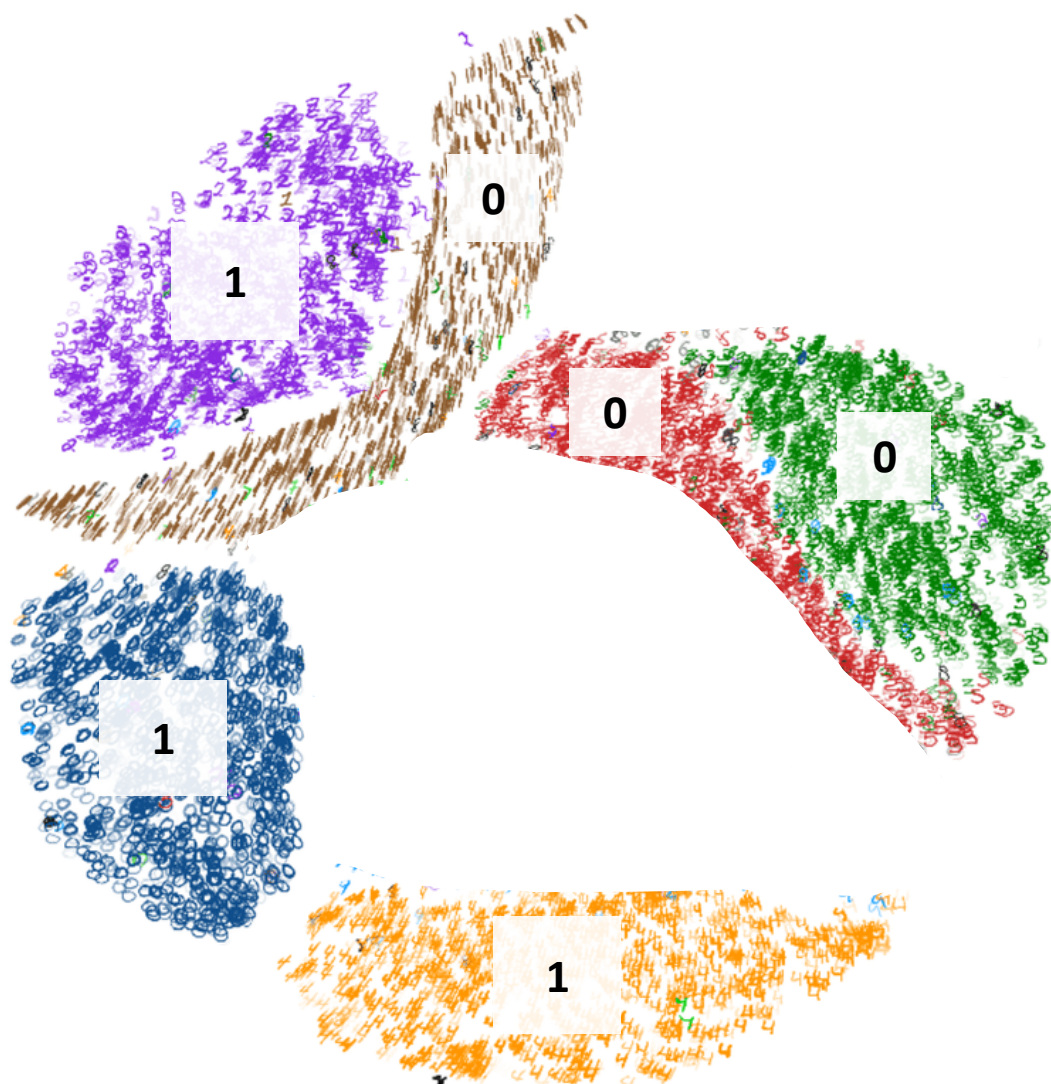
We consider a binary classification problem: **even/odd**.

Each task introduces a new couple of digits.

What we learned during previous tasks is still valid.

It's an Incremental Domain Learning Scenario.

MNIST dataset: Input drift scenario (CL-like)



 00 00	 11 11	 22 22	 33 33	 44 44	 55 55		
1 st task		2 nd task		3 rd task		4 th task	5 th task

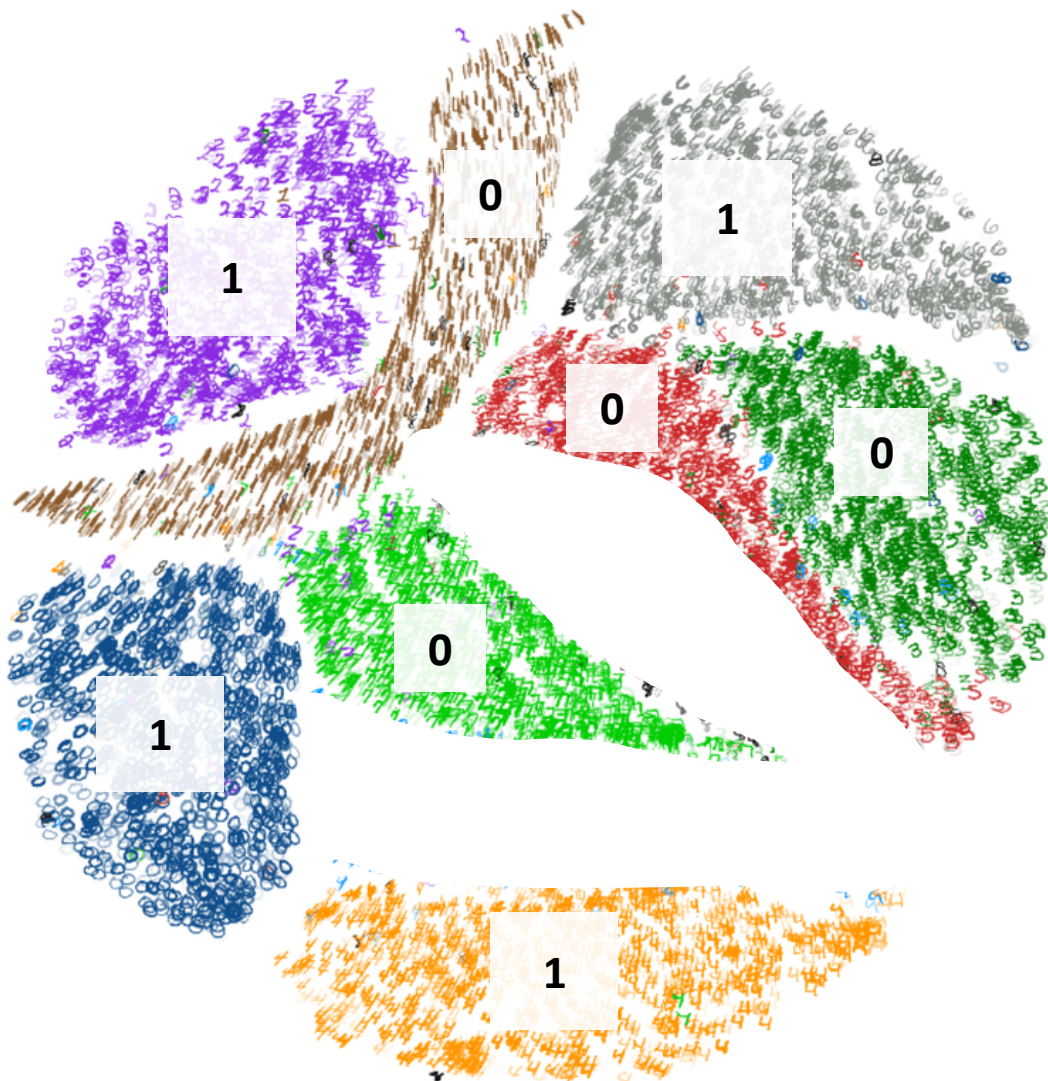
We consider a binary classification problem: **even/odd**.

Each task introduces a new couple of digits.

What we learned during previous tasks is still valid.

It's an Incremental Domain Learning Scenario.

MNIST dataset: Input drift scenario (CL-like)



 00 00	 11 11	 22 22	 33 33	 44 44	 55 55	 66 66	 77 77	
1 st task		2 nd task		3 rd task		4 th task		5 th task

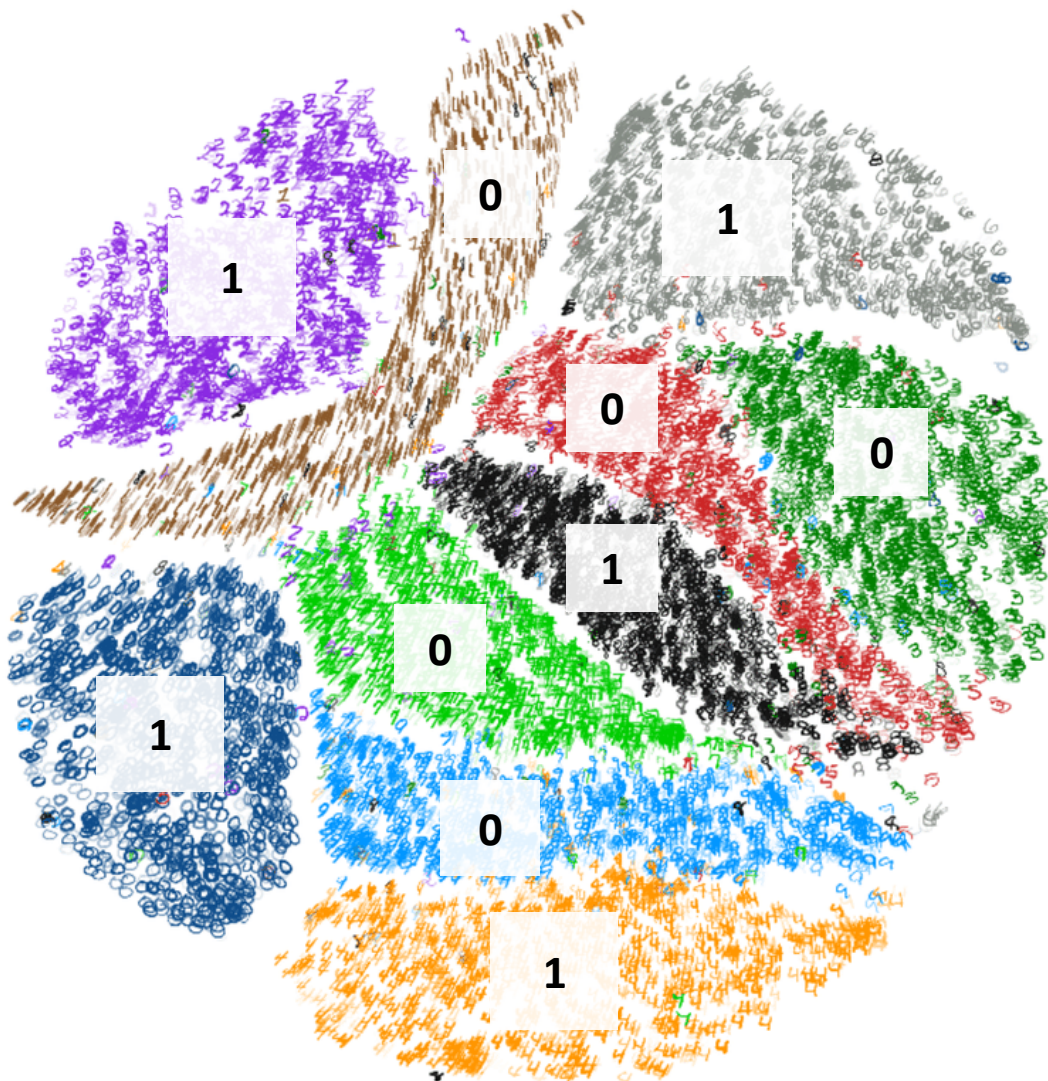
We consider a binary classification problem: **even/odd**.

Each task introduces a new couple of digits.

What we learned during previous tasks is still valid.

It's an Incremental Domain Learning Scenario.

MNIST dataset: Input drift scenario (CL-like)



 00 00	 11 11
1 st task	
 22 22	 33 33
2 nd task	
 44 44	 55 55
3 rd task	
 66 66	 77 77
4 th task	
 88 88	 99 99
5 th task	

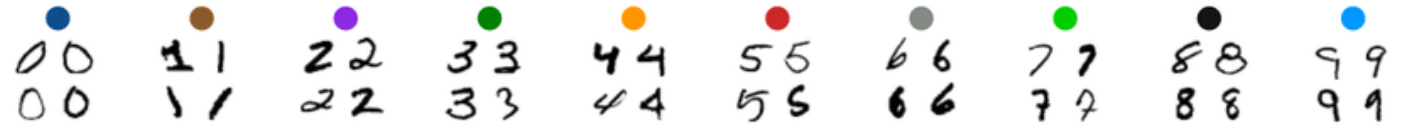
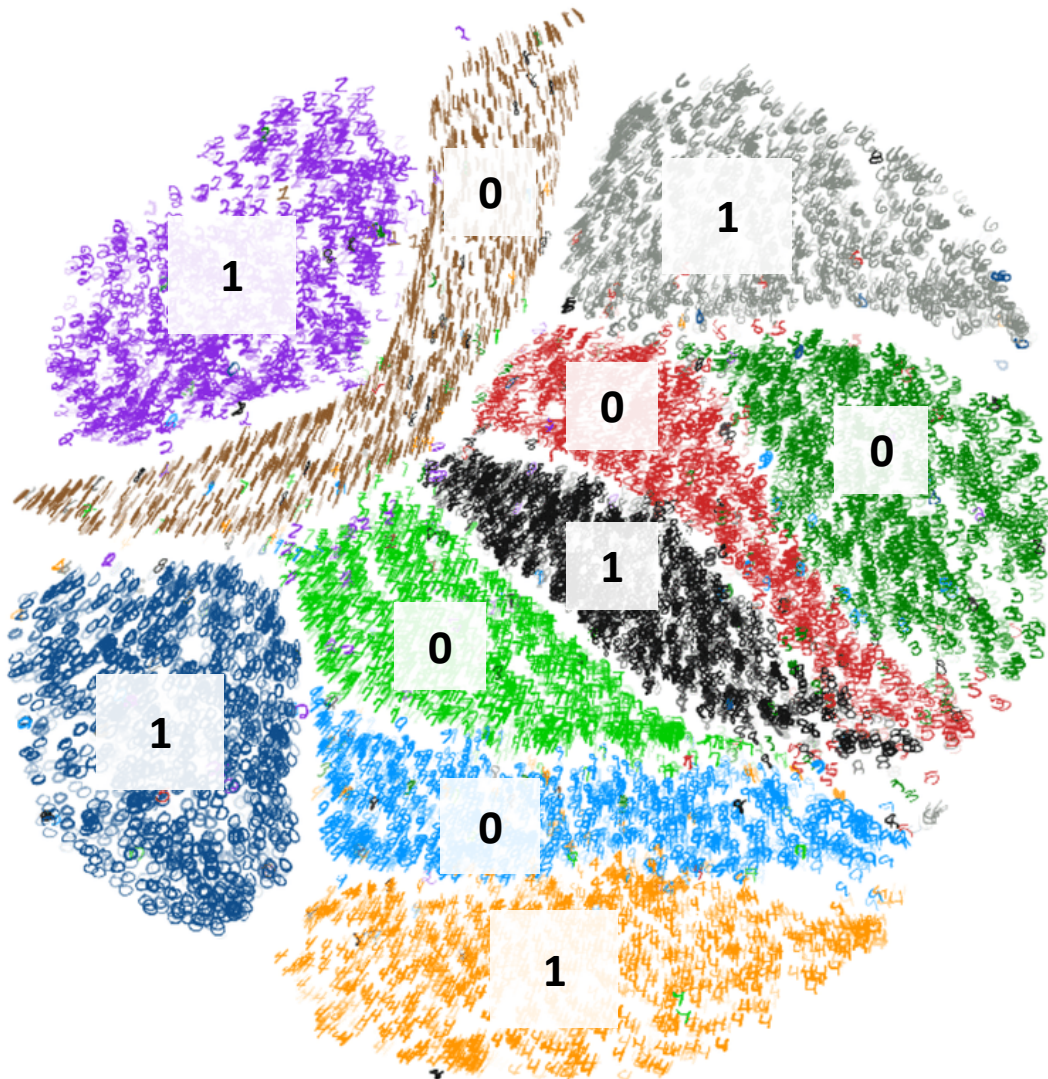
We consider a binary classification problem: **even/odd**.

Each task introduces a new couple of digits.

What we learned during previous tasks is still valid.

It's an Incremental Domain Learning Scenario.

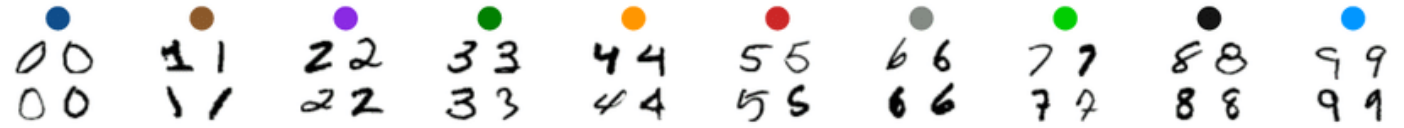
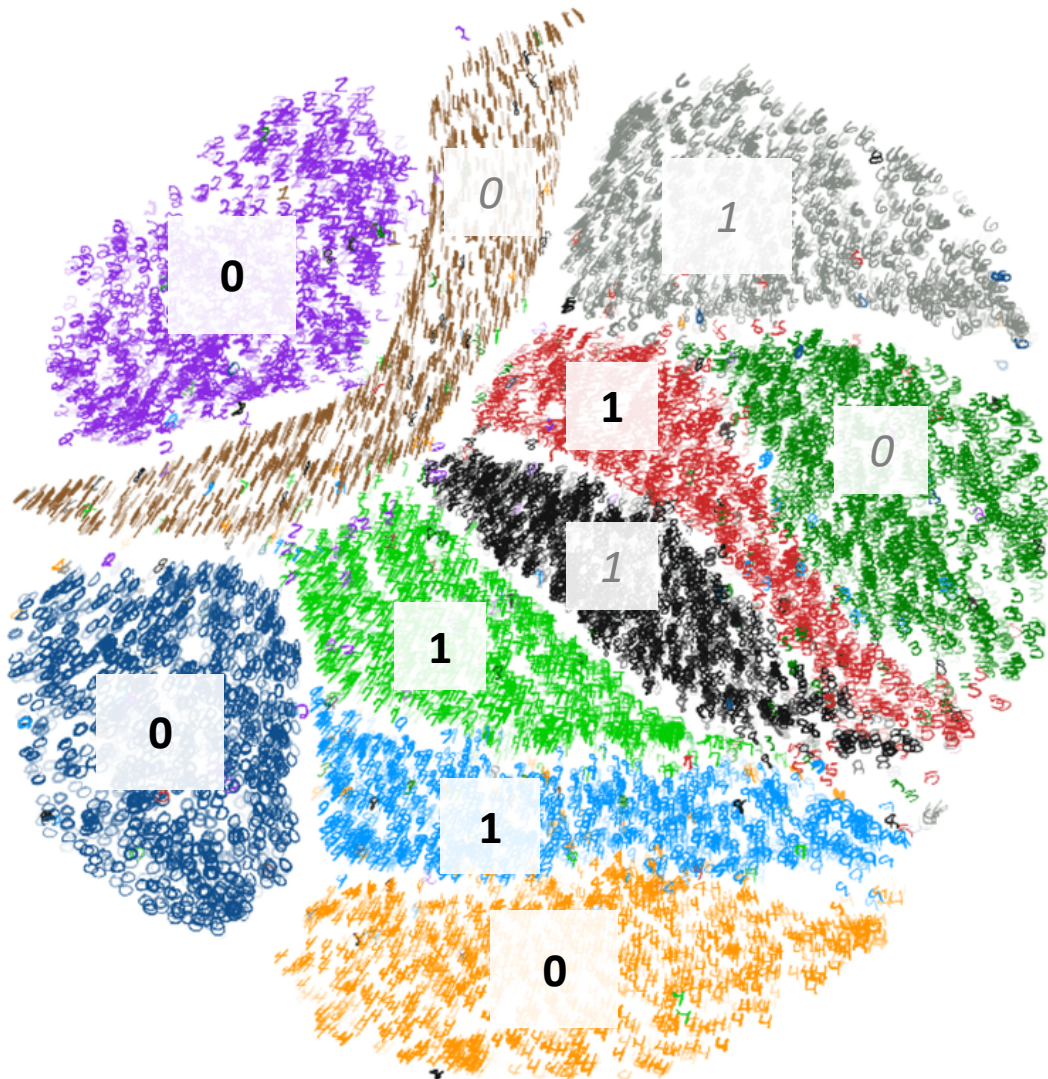
MNIST dataset: Contradictory drift scenario



We introduce five different binary problems (class boundaries) on the same input distributions, one for each task:

- **Even/odd (1st task)**

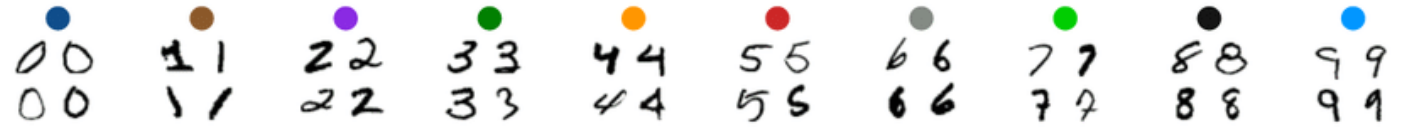
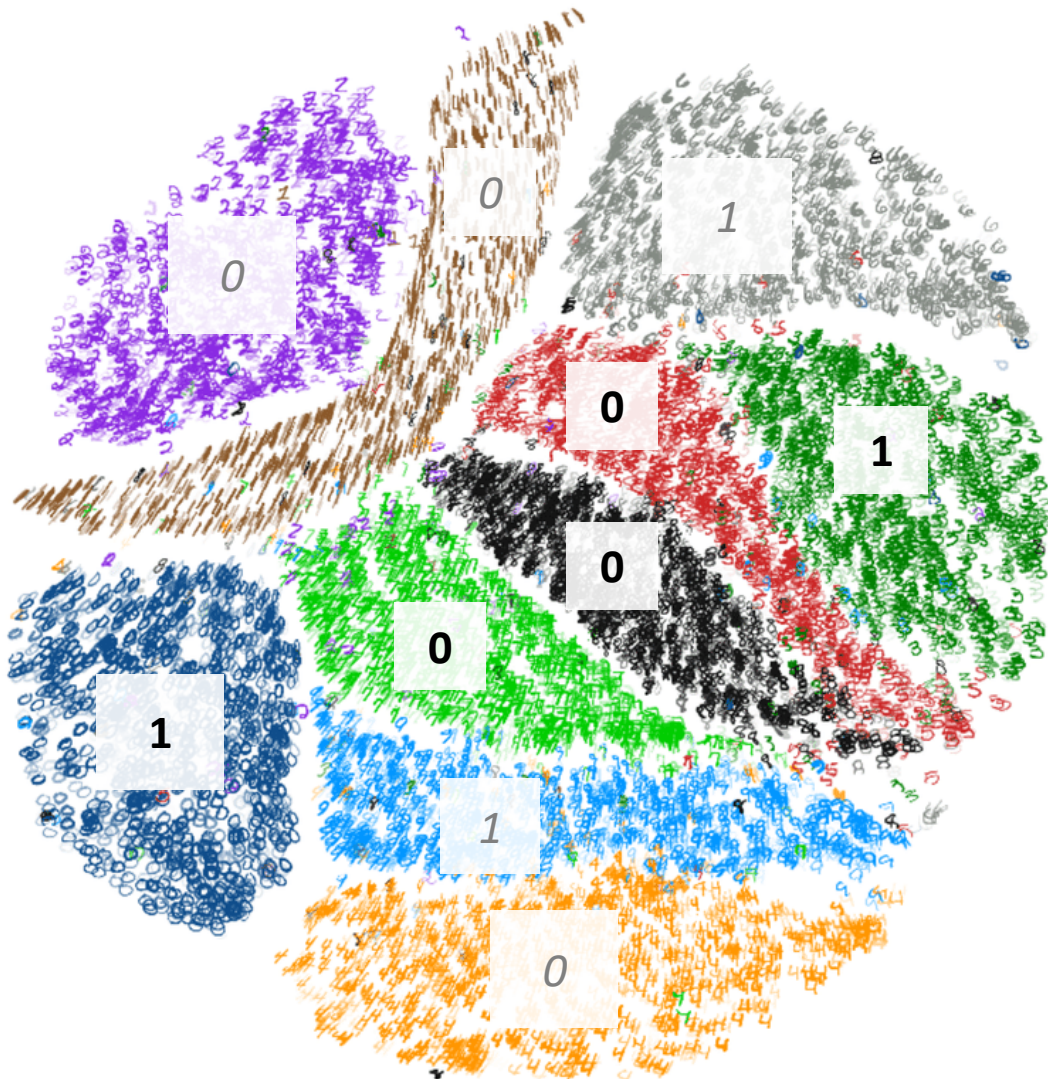
MNIST dataset: Contradictory drift scenario



We introduce five different binary problems (class boundaries) on the same input distributions, one for each task:

- *Even/odd* (1st task)
- **>4** (2nd task)

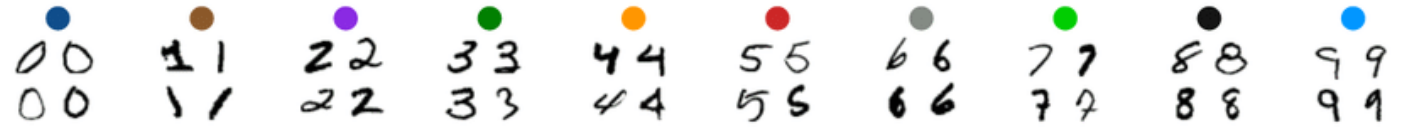
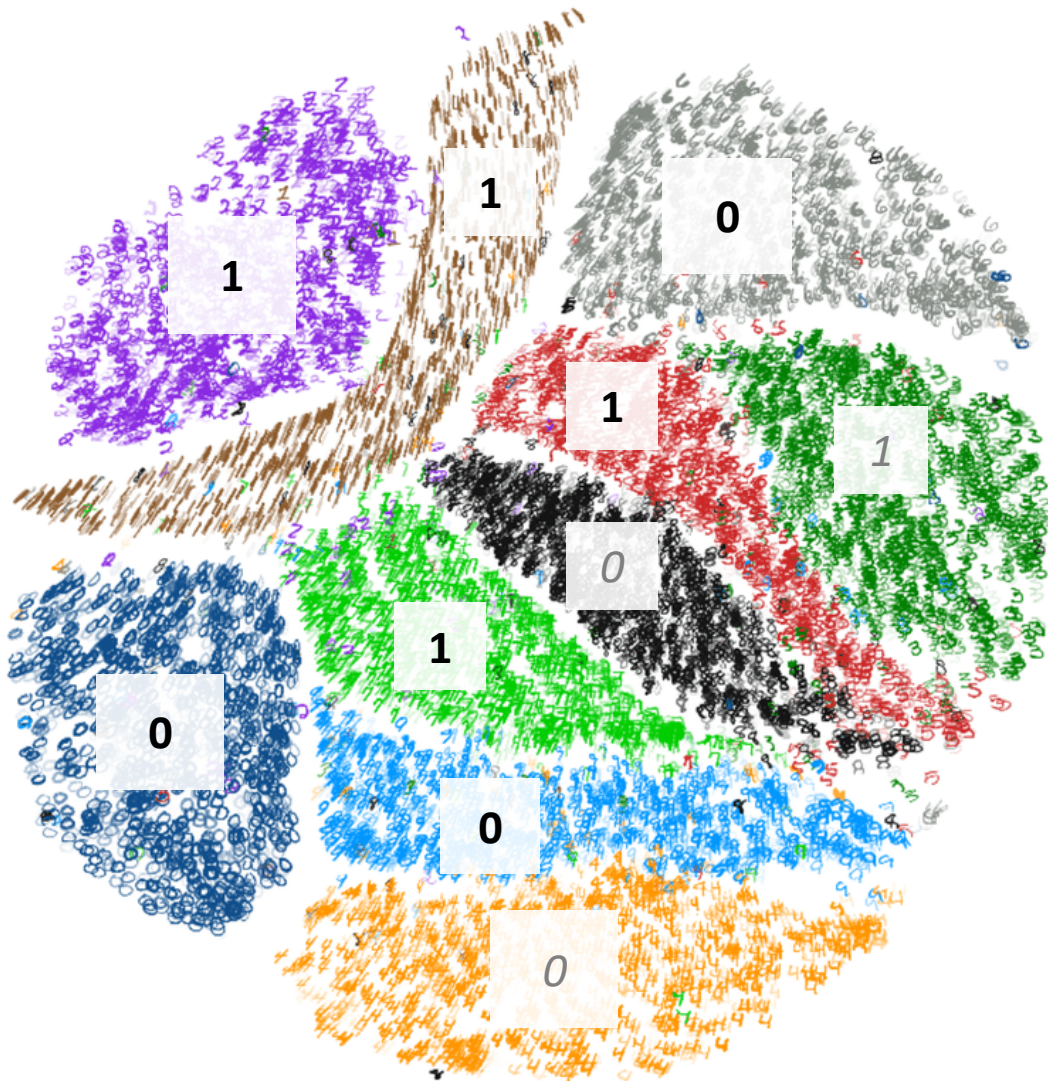
MNIST dataset: Contradictory drift scenario



We introduce five different binary problems (class boundaries) on the same input distributions, one for each task:

- *Even/odd* (1st task)
- *>4* (2nd task)
- **Multiple of 3** (3rd task)

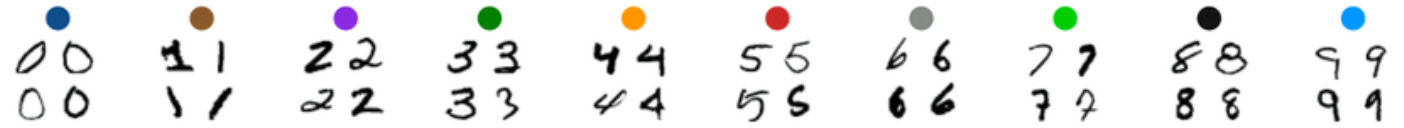
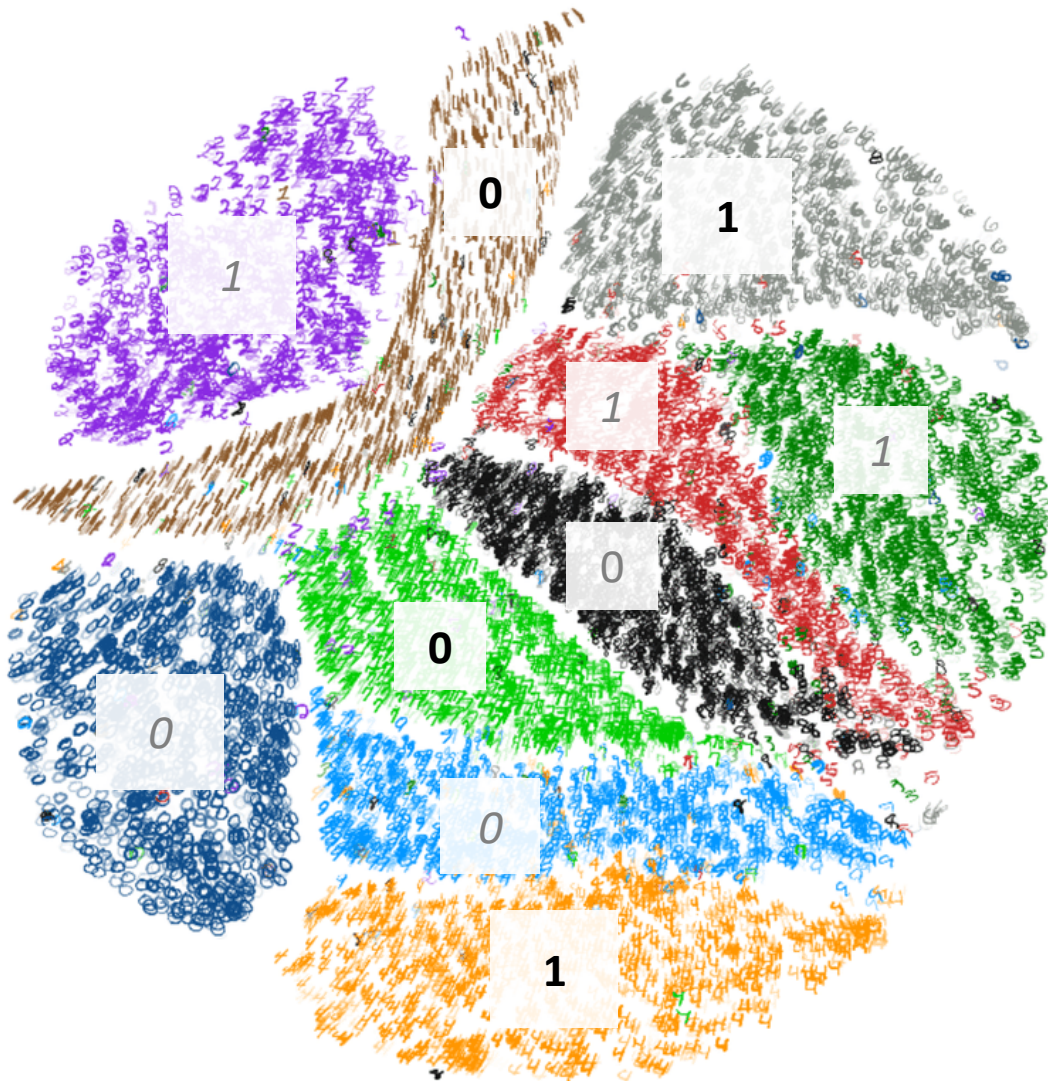
MNIST dataset: Contradictory drift scenario



We introduce five different binary problems (class boundaries) on the same input distributions, one for each task:

- *Even/odd* (1st task)
- *>4* (2nd task)
- *Multiple of 3* (3rd task)
- **Prime number including 1** (4th task)

MNIST dataset: Contradictory drift scenario



We introduce five different binary problems (class boundaries) on the same input distributions, one for each task:

- *Even/odd (1st task)*
- *>4 (2nd task)*
- *Multiple of 3 (3rd task)*
- *Prime number including 1 (4th task)*
- **Included in range [2;6] (5th task)**

Two types of tests:

- **Prequential evaluation** using the training set.
It tests how fast the model learns and adapts to drifts. We use a **rolling window with a size of 1k**.
Can models quickly adapt to changes? Are they plastic?
- **Continual Learning evaluation** using a test set for each task.
It tests how much the models retain the previous tasks' predictive ability when learning a new task.
Are models stable?
Can they remember the past tasks even if they contradict the current?

We use Cohen's Kappa as the performance metric since labels may be unbalanced.

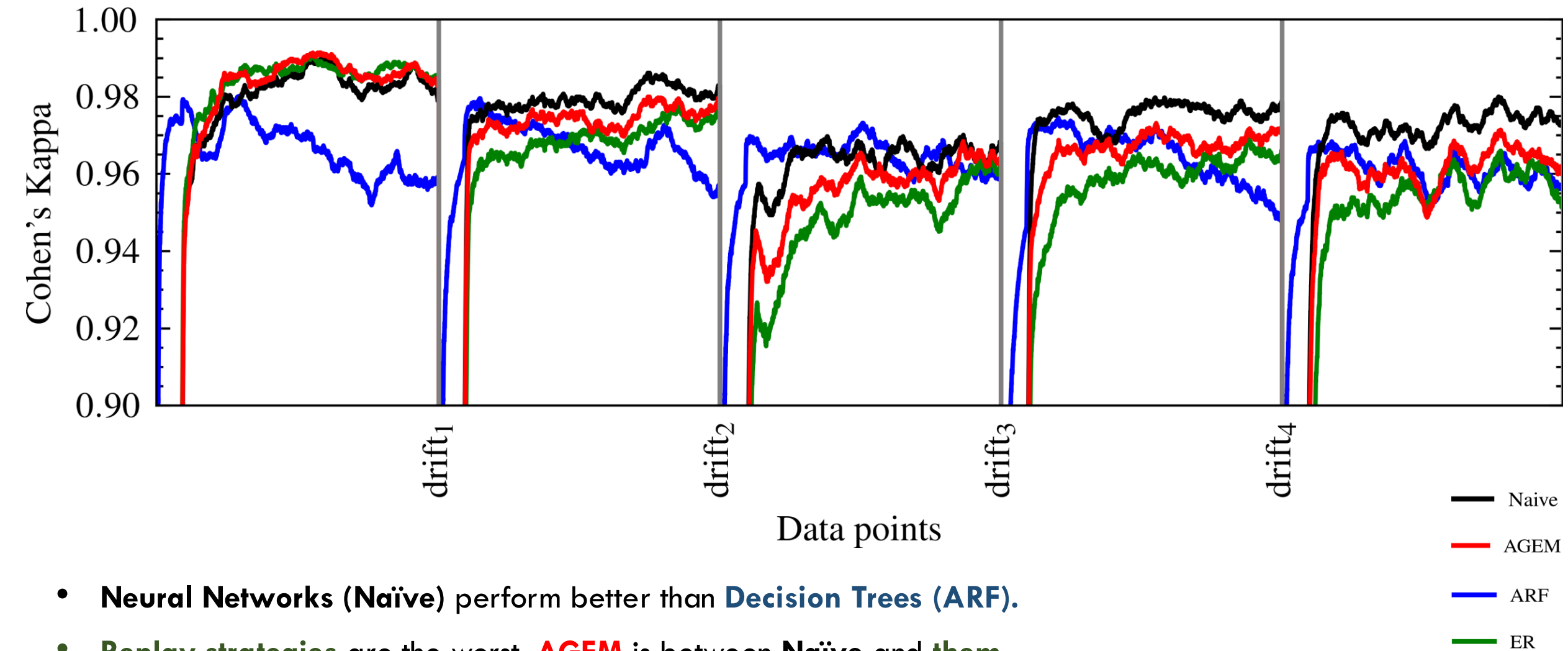
Experimental setting

- To conduct a fair comparison, we consider:
 - **ARF**: implemented using the default parameters of River.
 - **OCL** strategies applied to a Simple Neural Network with one layer of size 512.
For simplicity, we notify the OCL strategies when the task changes. Mini-batches contain 10 images. During prequential evaluation, inference is performed on each image, training on mini-batches containing ten images

In our experiments, we reduced the dimensionality to 30 using **UMAP** to ensure fairness between the Statistical ML (ARF) and the Neural Network models (OCL).



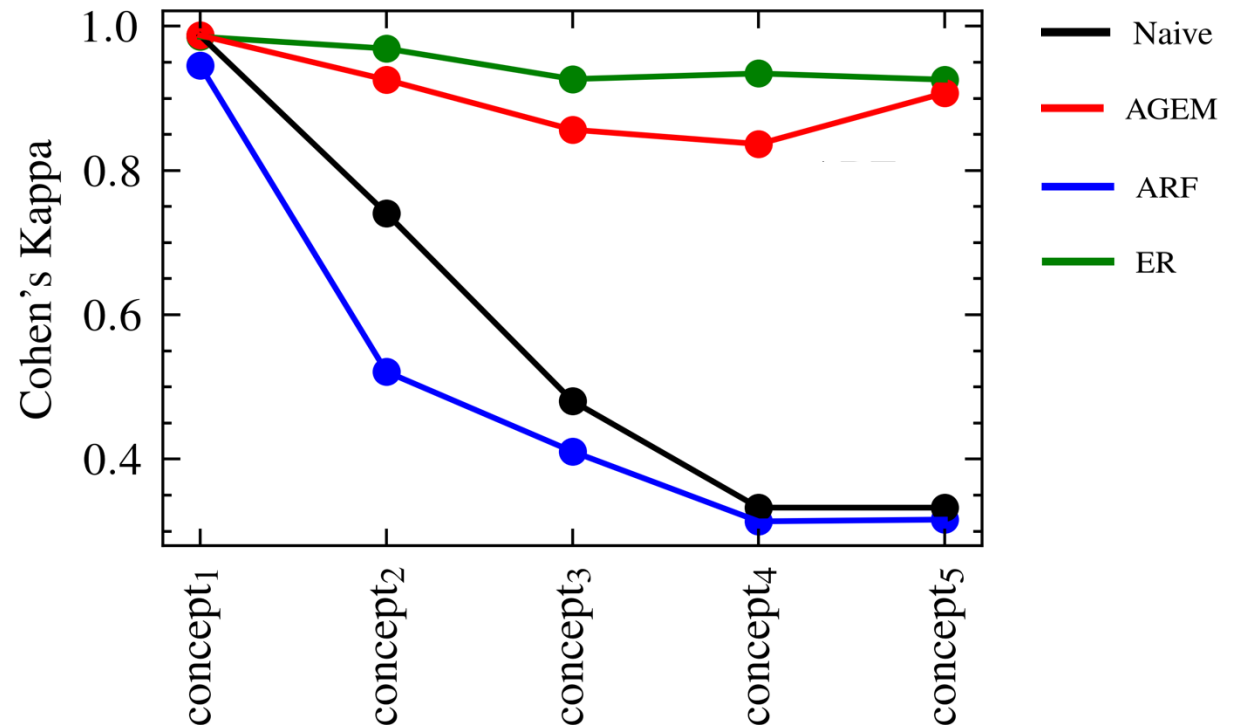
Input drift scenario: Prequential evaluation



- **Neural Networks (Naive)** perform better than **Decision Trees (ARF)**.
- **Replay strategies** are the worst. **AGEM** is between **Naive** and **them**.

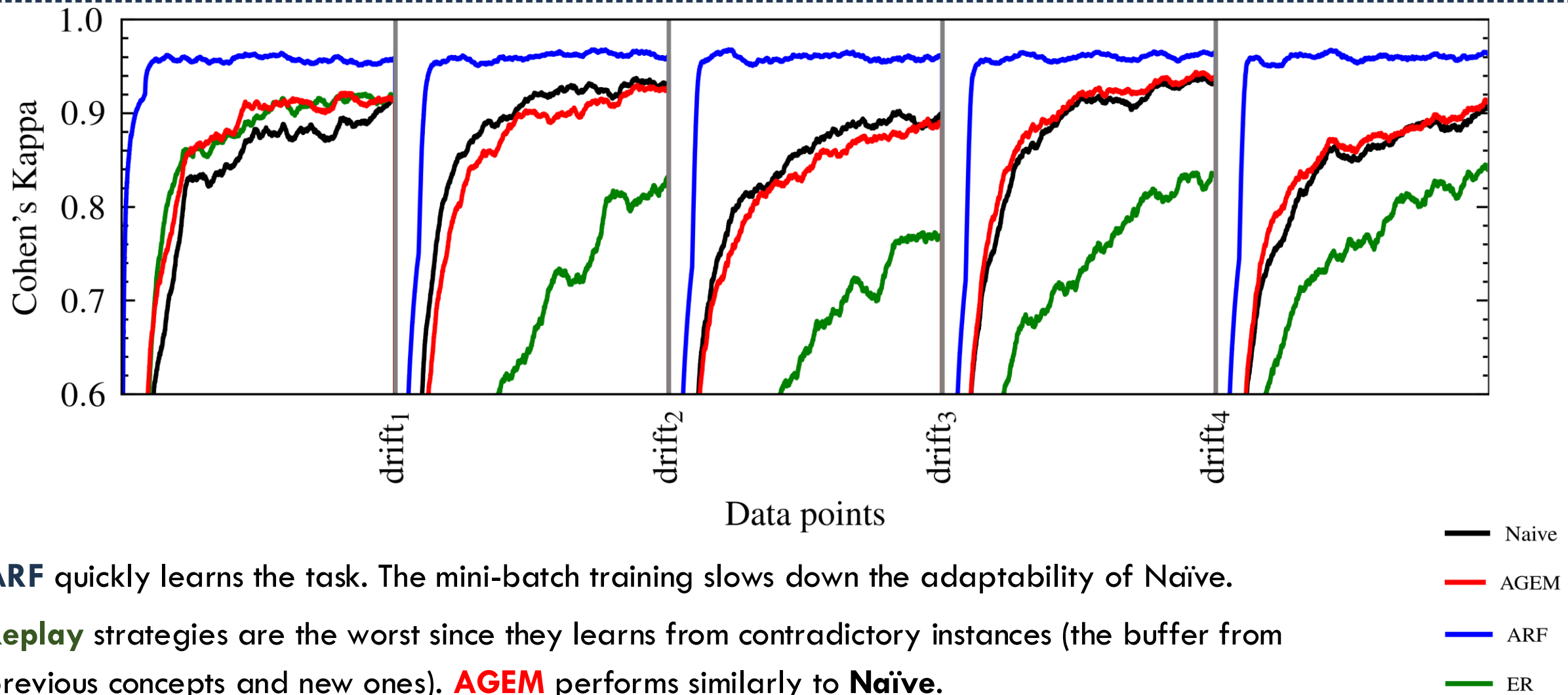
Input drift scenario: CL evaluation

	AVG	BWT
ER	0,94	-0,04
AGEM	0,89	-0,13
Naive	0,46	-0,77
ARF	0,41	-0,81



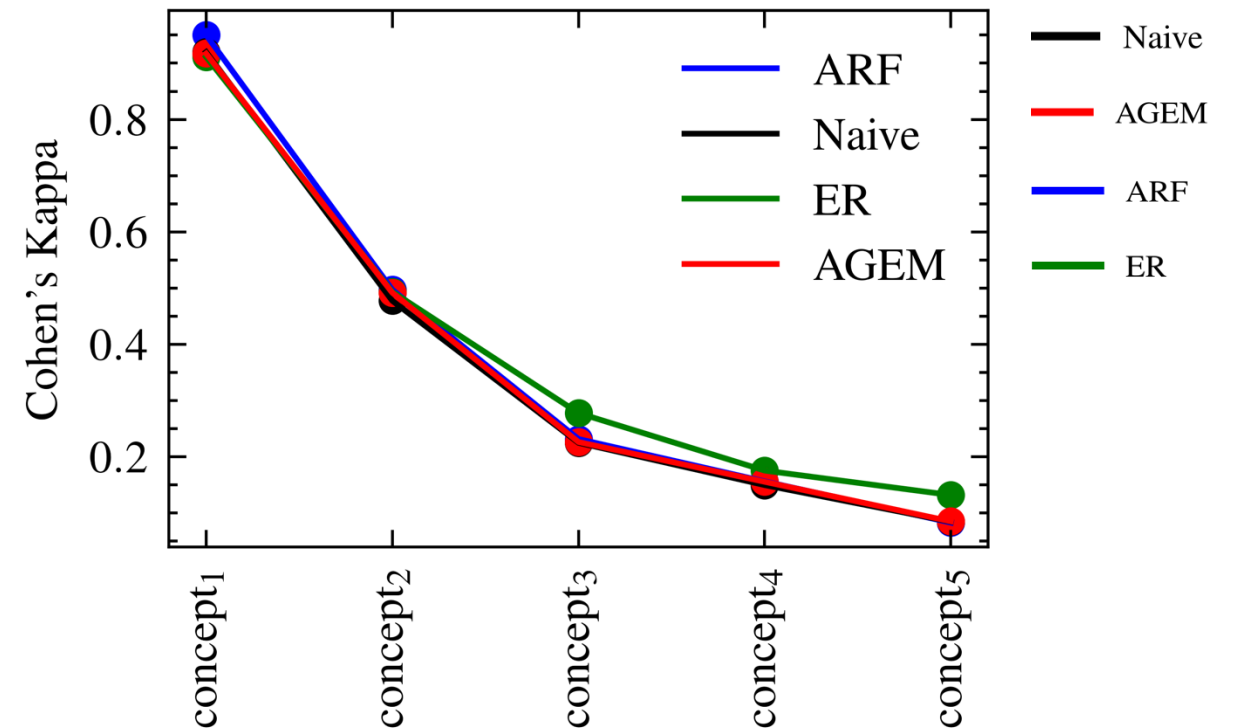
- **Replay strategies** can mitigate forgetting and perform well.
→ Avoiding forgetting slows down the plasticity of the new task.
- **AGEM** performs slightly worse but achieves good performance.
- **ARF** forgets the previous tasks.

Contradictory drift scenario: Prequential evaluation



Input drift scenario: CL evaluation

	AVG	BWT
ER	0,27	-0,85
ARF	0,24	-1,06
AGEM	0,24	-0,99
Naïve	0,24	-1



Replay attempts to mitigate forgetting but struggles to perform effectively.

→ CL strategies are not effective when the drifts are contradictory.

However, at least **they** obtain slightly better performance than the others.

→ Avoiding forgetting slows down the plasticity of the new task.

Conclusion

- SML focuses on the current concept and ignores the forgetting problem.
- CL is designed to handle scenarios where each new task introduces a new subproblem defined on a distinct feature subspace. Tasks are not contradictory.
- When tasks are contradictory, forgetting is impossible, and CL strategies are ineffective.
- Finding the trade-off between stability and plasticity slows down the plasticity on the current task. Replay ones are the most CL effective strategies and are the ones that perform worse in prequential evaluation.
- SML models forget the previous tasks after both types of drift.



Thank you!

